



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FCFM

FACULTAD DE CIENCIAS FÍSICO MATEMÁTICAS

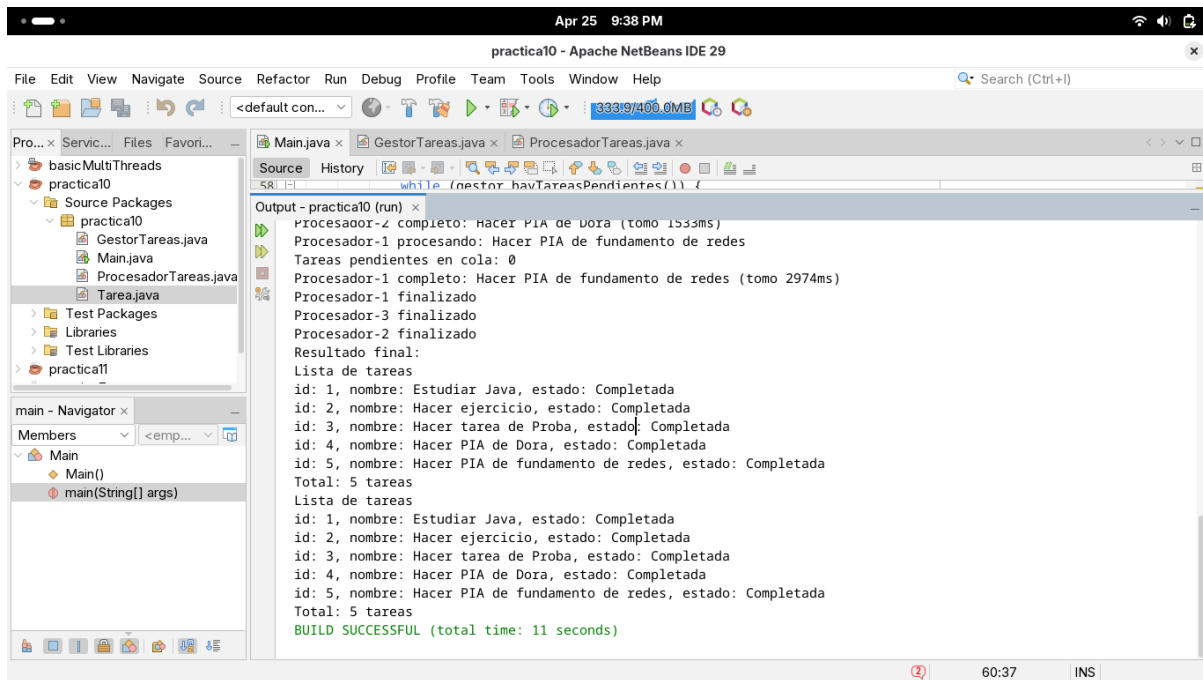
Laboratorio de programación orientado a objetos Practica 10

Alumno: Julio Antonio Blanco Rufino.

Matricula: 2003632.

Profesora: Jorge Alberto Islas Pineda.

Output:



run:

Gestor de tareas

tarea: Estudiar Java Agregada exitosamente

tarea: Hacer ejercicio Agregada exitosamente

tarea: Hacer tarea de Proba Agregada exitosamente

tarea: Hacer PIA de Dora Agregada exitosamente

tarea: Hacer PIA de fundamento de redes Agregada exitosamente

Error: ta existe una tarea con el nombre 'Estudiar Java'

Lista de tareas

id: 1, nombre: Estudiar Java, estado: Pendiente

id: 2, nombre: Hacer ejercicio, estado: Pendiente

id: 3, nombre: Hacer tarea de Proba, estado: Pendiente

id: 4, nombre: Hacer PIA de Dora, estado: Pendiente

id: 5, nombre: Hacer PIA de fundamento de redes, estado: Pendiente

Total: 5 tareas

Tarea encontrada: id: 3, nombre: Hacer tarea de Proba, estado: Pendiente

Estadísticas

ArrayList size: 5

HashMap size: 5

HashSet size: 5

LinkedList (cola) size: 5

Procesador-1 iniciado...

Procesador-3 iniciado...

Procesador-2 iniciado...

Procesador-1 procesando: Estudiar Java

Tareas pendientes en cola: 4

Tareas pendientes en cola: 4
Tareas pendientes en cola: 4
Procesador-1 completo: Estudiar Java (tomo 1885ms)
Procesador-2 procesando: Hacer ejercicio
Tareas pendientes en cola: 3
Tareas pendientes en cola: 3
Tareas pendientes en cola: 3
Tareas pendientes en cola: 3
Tareas pendientes en cola: 3
Procesador-2 completo: Hacer ejercicio (tomo 2420ms)
Procesador-3 procesando: Hacer tarea de Proba
Tareas pendientes en cola: 2
Tareas pendientes en cola: 2
Tareas pendientes en cola: 2
Tareas pendientes en cola: 2
Tareas pendientes en cola: 2
Procesador-3 completo: Hacer tarea de Proba (tomo 2440ms)
Procesador-2 procesando: Hacer PIA de Dora
Tareas pendientes en cola: 1
Tareas pendientes en cola: 1
Tareas pendientes en cola: 1
Procesador-2 completo: Hacer PIA de Dora (tomo 1533ms)
Procesador-1 procesando: Hacer PIA de fundamento de redes
Tareas pendientes en cola: 0
Procesador-1 completo: Hacer PIA de fundamento de redes (tomo 2974ms)
Procesador-1 finalizado
Procesador-3 finalizado
Procesador-2 finalizado
Resultado final:
Lista de tareas
id: 1, nombre: Estudiar Java, estado: Completada
id: 2, nombre: Hacer ejercicio, estado: Completada
id: 3, nombre: Hacer tarea de Proba, estado: Completada
id: 4, nombre: Hacer PIA de Dora, estado: Completada
id: 5, nombre: Hacer PIA de fundamento de redes, estado: Completada
Total: 5 tareas
Lista de tareas
id: 1, nombre: Estudiar Java, estado: Completada
id: 2, nombre: Hacer ejercicio, estado: Completada
id: 3, nombre: Hacer tarea de Proba, estado: Completada
id: 4, nombre: Hacer PIA de Dora, estado: Completada

id: 5, nombre: Hacer PIA de fundamento de redes, estado: Completada

Total: 5 tareas

BUILD SUCCESSFUL (total time: 11 seconds)

Main:

/*

- Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Main.java to edit this template

*/ package practica10;

/** *

- @author alba

*/

```
public class Main { public static void main(String[] args) {  
    System.out.println("Gestor de tareas");
```

```
    GestorTareas gestor = new GestorTareas();
```

```
    Tarea tarea1 = new Tarea(1, "Estudiar Java", "Pendiente");
```

```
    Tarea tarea2 = new Tarea(2, "Hacer ejercicio", "Pendiente");
```

```
    Tarea tarea3 = new Tarea(3, "Hacer tarea de Proba", "Pendiente");
```

```
    Tarea tarea4 = new Tarea(4, "Hacer PIA de Dora", "Pendiente");
```

```
    Tarea tarea5 = new Tarea(5, "Hacer PIA de fundamento de redes",  
    "Pendiente");
```

```
    gestor.agregarTarea(tarea1);
```

```
    gestor.agregarTarea(tarea2);
```

```
    gestor.agregarTarea(tarea3);
```

```
    gestor.agregarTarea(tarea4);
```

```
    gestor.agregarTarea(tarea5);
```

```
Tarea tareaDuplicada = new Tarea(6, "Estudiar Java", "Pendiente");
gestor.agregarTarea(tareaDuplicada);
```

```
gestor.mostrarTareas();
```

```
Tarea encontrada = gestor.buscarPorId(3);
if (encontrada != null) {
    System.out.println("Tarea encontrada: " + encontrada);
}
```

```
gestor.mostrarEstadisticas();
```

```
ProcesadorTareas procesador1 = new
ProcesadorTareas(gestor.getCola(), "Procesador-1");
ProcesadorTareas procesador2 = new
ProcesadorTareas(gestor.getCola(), "Procesador-2");
ProcesadorTareas procesador3 = new
ProcesadorTareas(gestor.getCola(), "Procesador-3");
```

```
Thread hilo1 = new Thread(procesador1);
Thread hilo2 = new Thread(procesador2);
Thread hilo3 = new Thread(procesador3);
```

```
hilo1.start();
hilo2.start();
hilo3.start();
```

```
try {
    while (gestor.hayTareasPendientes()) {
        Thread.sleep(500);
        System.out.println("Tareas pendientes en cola: " +
gestor.getCola().size());
    }

    Thread.sleep(2000);

} catch (InterruptedException e) {
```

```

        System.out.println("Main thread interrumpido");
    }

    procesador1.detener();
    procesador2.detener();
    procesador3.detener();

    try {
        hilo1.join();
        hilo2.join();
        hilo3.join();
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    System.out.println("Resultado final:");
    gestor.mostrarTareas();

    gestor.guardarArchivo();

    GestorTareas nuevoGestor = new GestorTareas();
    nuevoGestor.leerArchivo();
    nuevoGestor.mostrarTareas();
}

}

GestorTareas:
/*

    • Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-
      default.txt to change this license
    • Click
      nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to
      edit this template

*/ package practica10;

```

```
import java.io.BufferedReader; import java.io.BufferedWriter; import
java.io.File; import java.io.FileReader; import java.io.FileWriter; import
java.io.IOException; import java.util.ArrayList; import java.util.HashMap;
import java.util.HashSet; import java.util.Iterator; import
java.util.LinkedList;
```

```
/** *
```

- @author alba

```
*/
```

```
public class GestorTareas { ArrayList lista; HashMap<Integer,Tarea>
mapa; HashSet set; LinkedList cola;
```

```
private File carpeta = new File("Tareas"); private File archivo = new
File(carpeta, "tarea.txt");
```

```
public GestorTareas(){ this.lista = new ArrayList<>(); this.mapa = new
HashMap<>(); this.set = new HashSet<>(); this.cola = new
LinkedList<>(); this.crearCarpetasYArchivo(); }
```

```
public void crearCarpetasYArchivo(){ try{ if(!carpeta.exists()){
carpeta.mkdirs(); } if(!archivo.exists()){ archivo.createNewFile(); } }
catch(IOException e){ System.out.println("Error: " + e.getMessage()); } }
```

```
public void agregarTarea(Tarea tarea){
if(mapa.containsKey(tarea.getId())){ System.out.println("Error: ya existe
una tarea con ID " + tarea.getId()); return; }
```

```
    if(set.contains(tarea.getNombre())){
        System.out.println("Error: ta existe una tarea con el nombre
""+tarea.getNombre()+""");
        return;
    }
    lista.add(tarea);
    mapa.put(tarea.getId(), tarea);
    set.add(tarea.getNombre());
    cola.add(tarea);
```

```
        System.out.println("tarea: " + tarea.getNombre() + " Agregada exitosamente");
```

```
    }
```

```
    public void guardarArchivo(){ try{ BufferedWriter bwrite = new  
        BufferedWriter(new FileWriter(archivo)); for(Tarea tarea: lista){  
        bwrite.write(tarea.toFileString()); bwrite.newLine(); } bwrite.close(); }  
    catch(IOException e){ System.out.println("Error " + e.getMessage()); } }
```

```
    public void leerArchivo(){  
    try{  
        BufferedReader bread = new BufferedReader(new  
        FileReader(archivo));  
        lista.clear();  
        String linea;  
        String[] argumentos;  
        int id;  
        String nombre;  
        String estado;
```

```
        while((linea = bread.readLine()) != null){  
            argumentos = linea.split(",");  
            id = Integer.parseInt(argumentos[0]);  
            nombre = argumentos[1];  
            estado = argumentos[2];
```

```
            Tarea tarea = new Tarea(id, nombre, estado);  
            lista.add(tarea);
```

```
        }  
        bread.close();
```

```
    }  
    catch(IOException e){  
        System.out.println("Error: " + e.getMessage());  
    }  
}
```

```
public void mostrarTareas() {
    System.out.println("Lista de tareas");
    if (lista.isEmpty()) {
        System.out.println("No hay tareas registradas");
        return;
    }

    for (Tarea tarea : lista) {
        System.out.println(tarea);
    }
    System.out.println("Total: " + lista.size() + " tareas");
}
```

```
public Tarea buscarPorId(int id) {
    return mapa.get(id);
}

public void mostrarTareasConIterator() {
    System.out.println("Tareas usando Iterator:");
    Iterator<Tarea> iterator = lista.iterator();
    while (iterator.hasNext()) {
        System.out.println(iterator.next());
    }
}
```

```
public LinkedList<Tarea> getCola() {
    return cola;
}
```

```
public boolean hayTareasPendientes() {
    return !cola.isEmpty();
}
```

```
public void mostrarEstadisticas() {
    System.out.println("Estadisticas");
    System.out.println("ArrayList size: " + lista.size());
    System.out.println("HashMap size: " + mapa.size());
    System.out.println("HashSet size: " + set.size());
}
```

```

        System.out.println("LinkedList (cola) size: " + cola.size());
    }

}

```

Tarea:

```
/*
```

- Click [nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt](#) to change this license
- Click [nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java](#) to edit this template

```
*/ package practica10;
```

```
/** *
```

- @author alba

```
*/
```

```
public class Tarea { private int id; private String nombre; private String estado;
```

```

    public Tarea(int id, String nombre, String estado){
        this.id = id;
        this.nombre = nombre;
        this.estado = estado;
    }

```

```

    public String toFileString(){
        return getId()+","+getNombre()+","+getEstado();
    }

```

@Override

```

    public String toString(){
        return "id: " + getId() +
            ", nombre: " + getNombre() +

```

```
        ", estado: " + getEstado());  
    }
```

```
/**  
 * @return the id  
 */  
public int getId() {  
    return id;  
}
```

```
/**  
 * @return the nombre  
 */  
public String getNombre() {  
    return nombre;  
}
```

```
/**  
 * @return the estado  
 */  
public String getEstado() {  
    return estado;  
}
```

```
/**  
 * @param estado the estado to set  
 */  
public void setEstado(String estado) {  
    this.estado = estado;  
}
```

```
}
```

Procesador de tareas:

```
/*
```

- Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change this license
- Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template

```
*/ package practica10;
```

```
import java.util.LinkedList;
```

```
/** *
```

- @author alba

```
*/
```

```
public class ProcesadorTareas implements Runnable { private
LinkedList cola; private String nombreProcesador; private boolean
ejecutando;
```

```
public ProcesadorTareas(LinkedList<Tarea> cola, String
nombreProcesador) {
    this.colas = cola;
    this.nombreProcesador = nombreProcesador;
    this.ejecutando = true;
}
```

```
public void detener() {
    this.ejecutando = false;
}
```

```
@Override
```

```
public void run() {
    System.out.println(nombreProcesador + " iniciado...");
```

```
while (ejecutando) {
    synchronized (cola) {
        if (!cola.isEmpty()) {
            Tarea tarea = cola.poll();
            System.out.println(nombreProcesador + " procesando: " +
```

```

tarea.getNombre());

        try {
            int tiempoProcesamiento = (int) (Math.random() * 2000) +
1000;
            Thread.sleep(tiempoProcesamiento);

            tarea.setEstado("Completada");
            System.out.println(nombreProcesador + " completo: " +
tarea.getNombre()
                        + " (tomo " + tiempoProcesamiento + "ms)");

        } catch (InterruptedException e) {
            System.out.println("ERROR: " + nombreProcesador + " fue
interrumpido");
            Thread.currentThread().interrupt();
        }
    }

    try {
        Thread.sleep(100);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}

System.out.println(nombreProcesador + " finalizado");
}

}

```